

LAPORAN PRAKTIKUM PEKAN 1

PEMOGRAMAN BERORIENTASI OBJEK



Disusun oleh:

ANNAFI AL GHIFARI

2511533013

Dosen pengampu

Wahyudi. Dr., S.T,M.T

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2025

DAFTAR ISI

KATA PENGANTAR	Error! Bookmark not defined.
BAB 1	Error! Bookmark not defined.
PENDAHULUAN	3
CONTOH KASUS PENERAPAN & KETERKAITAN TAHAPAN LIFECYCLE	Error!
Bookmark not defined.	
 Tabel Keterkaitan Alur Kasus dengan Lifecycle Data Science	Error! Bookmark not defined.
DAFTAR PUSTAKA.....	Error! Bookmark not defined.

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mendorong penggunaan aplikasi dalam berbagai bidang, salah satunya adalah sistem perbankan. Sistem perbankan membutuhkan pengelolaan data yang terstruktur agar informasi rekening, saldo, serta transaksi dapat dikelola dengan baik. Dalam pemrograman, konsep **Object-Oriented Programming (OOP)** dapat digunakan untuk membuat sistem sederhana yang mampu merepresentasikan rekening dan aktivitas perbankan.

Pada praktikum ini dibuat sebuah **sistem perbankan mini menggunakan bahasa pemrograman Java**. Program memiliki beberapa fitur, seperti membuka rekening baru, setor tunai, tarik tunai, melihat informasi rekening, serta mengganti akun. Sistem juga menerapkan validasi transaksi, seperti minimal setor dan tarik tunai sebesar Rp10.000 serta memastikan saldo tidak menjadi minus.

Program kemudian dikembangkan menggunakan struktur data **Collection**, khususnya `ArrayList<Rekening>`, untuk mengatasi keterbatasan sistem yang sebelumnya hanya dapat menyimpan satu rekening. Dengan penggunaan `ArrayList`, beberapa objek rekening dapat disimpan sekaligus dan pengguna dapat memilih rekening yang ingin digunakan melalui fitur **Ganti Akun**. Praktikum ini diharapkan dapat membantu memahami penerapan konsep OOP, Collection, `ArrayList`, percabangan, perulangan, dan input pengguna dalam pembuatan program Java.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami penerapan konsep Object-Oriented Programming (OOP) dalam bahasa pemrograman Java.
2. Membuat program sistem perbankan sederhana yang dapat mengelola data dan transaksi rekening.
3. Menerapkan struktur data Collection menggunakan `ArrayList<Rekening>` untuk menyimpan beberapa rekening.
4. Menerapkan fitur Buka Rekening, Setor Tunai, Tarik Tunai, Cek Informasi Rekening, dan Ganti Akun.
5. Menerapkan validasi input dan transaksi agar program dapat berjalan dengan baik dan mencegah saldo menjadi minus.

1.3 Manfaat

Manfaat dari praktikum ini adalah:

1. Menambah pemahaman mengenai penggunaan **class, object, constructor, dan method** dalam Java.
2. Memahami cara menggunakan **ArrayList** untuk menyimpan dan mengelola banyak objek.

PEMBAHASAN

DASAR TEORI

Double Linked List (DLL) salah satu jenis struktur data linked list yang terdiri dari sekumpulan node yang saling terhubung satu sama lain. Setiap node menyimpan data serta memiliki dua pointer, yaitu next dan prev. Pointer next digunakan untuk menunjuk ke node berikutnya, sedangkan pointer prev digunakan untuk menunjuk ke node sebelumnya. Dengan adanya dua pointer tersebut, hubungan antar node menjadi dua arah.

Karena dapat diakses dari dua arah, proses penelusuran data pada Double Linked List menjadi lebih fleksibel. Data tidak hanya dapat ditelusuri dari awal ke akhir, tetapi juga dapat ditelusuri kembali dari akhir ke awal. Hal ini berbeda dengan Single Linked List yang hanya memiliki satu pointer next, sehingga penelusuran data hanya bisa dilakukan satu arah saja. Pada Double Linked List, perpindahan antar node menjadi lebih mudah karena setiap node mengetahui node sebelum dan sesudahnya.

2.1 Langkah-Langkah Praktikum

2.1.1 Membuat project dan package

Sebelum membuat program, diawali dengan membuat package sebagai tempat penyimpanan file project yang akan dikerjakan. Package digunakan agar file Rekening.java dan mainRekening.java berada dalam satu Kelompok yang sama.

2.1.2 Membuat class rekening

Class Rekening menjadi dasar untuk membuat object rekening yang akan digunakan, class Rekening digunakan sebagai model untuk menyimpan data rekening. Terdapat atribut nomor rekening, nama pemilik, dan saldo.

2.1.3 Membuat constructor

Setelah itu, ditambahkan constructor pada class Rekening. Constructor digunakan untuk memberikan nilai awal pada project ketika rekening baru dibuat. Nilai yang dimasukkan berupa nomor rekening, nama pemilik, dan saldo awal. Constructor akan dijalankan ketika object Rekening akan dibuat

2.1.4 Membuat method pada class Rekening

Class Rekening kemudian dilengkapi dengan beberapa method untuk menjalankan fungsi.

2.1.5 Membuat class mainRekening

Setelah class Rekening selesai, dibuat class mainRekening sebagai tempat menjalankan program utama.

2.1.6 Membuat menu utama

Membuat menu utama yang berisi pilihan untuk membuka rekening baru, melakukan setor tunai, tarik tunai, melihat informasi rekening, mengganti akun, dan keluar dari program. Pengguna dapat memilih menu dengan memasukkan nomor sesuai pilihan yang tersedia.

2.1.7 Membuat rekening baru

Dalam menu pembuatan rekening baru, program menyimpan nomor rekening yang dimasukkan oleh user, menyimpan nama pemilik rekening, dan saldo yang ada pada rekening tersebut.

2.1.8 Melakukan setor dan tarik tunai

Pada menu setor tunai, pengguna memasukkan nominal yang ingin disetorkan dan program akan menambahkannya ke saldo rekening yang sedang aktif. Sementara itu, pada menu tarik tunai, program akan mengurangi saldo sesuai nominal yang dimasukkan dengan etap melakukan pengecekan agar saldo tidak menjadi minus.

2.1.9 Mengecek informasi rekening

Pada menu cek informasi, program akan menampilkan data dari rekening yang sedang aktif. Informasi yang ditampilkan berupa nomor rekening, nama pemilik, dan saldo yang dimiliki oleh rekening tersebut.

2.1.10 Menggunakan menu ganti akun

Menambahkan menu ganti akun dengan menambahkan method ganti akun juga yang berguna untuk user jika user menginginkan pergantian akun dalam program. Menu ganti akun digunakan method ArrayList agar data sebelumnya dan data setelahnya yang telah diinput dapat tersimpan.

2.1.11 Menguji program

Setelah semua kode program selesai dengan class serta method, maka selanjutnya adalah menguji kode tersebut apakah sesuai dengan kebutuhan user atau tidak. Dalam hal pengujian perlu menggunakan *problem solving* juga untuk mengetahui masalah apa yang terjadi pada kode program, jika kode program tidak sesuai dengan yang user inginkan.

2.1.12 Mengakhiri program

Program dapat dihentikan dengan memilih menu keluar. Setelah pilihan tersebut dipilih, program akan berhenti berjalan.

2.2 Tugas dan Latihan

2.2.1 Kode Program

```
package pekan1;

public class Rekening {

    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor, String nama, double saldoAwal) {

        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;

        System.out.println(
            "Rekening atas nama " + namaPemilik +
            " berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
```

```
if (nominal >= 10000) {

    saldo += nominal;

    System.out.println(
        "Setor tunai Rp" + nominal +
        " berhasil. Saldo saat ini: Rp" + saldo);

} else {

    System.out.println(
        "Gagal, nominal setor minimal Rp10.000!");
}
}

public void tarikTunai(double nominal) {

    if (nominal < 10000) {

        System.out.println(
            "Gagal, nominal tarik tunai minimal Rp10.000!");

    } else if (nominal > saldo) {

        System.out.println(
            "Gagal, saldo tidak mencukupi, saldo Anda : Rp" + saldo);

    } else {

        saldo -= nominal;

        System.out.println(
            "Tarik tunai Rp" + nominal +
            " berhasil. Saldo saat ini: Rp" + saldo);

    }
}

public void cekInformasi() {

    System.out.println("\n--- INFO REKENING ---");
```

```

        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir : Rp" + saldo);
        System.out.println("-----");
    }

    public void gantiAkun(String nomor, String nama, double saldoAwal) {

        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;

        System.out.println(
            "Rekening atas nama " + namaPemilik +
            " berhasil diganti dengan saldo Rp" + saldo);
    }
}

```

2.2.2 Penjelasan

```
package pekan1;
```

```
public class Rekening {
```

Kode tersebut digunakan untuk menentukan bahwa class Rekening berada di dalam package pekan1. Class Rekening digunakan sebagai tempat untuk menyimpan data dan fungsi yang berkaitan dengan rekening bank.

```
String nomorRekening;
String namaPemilik;
double saldo;
```

Terdapat tiga atribut yang digunakan untuk menyimpan data rekening, yaitu nomorRekening untuk menyimpan nomor rekening, namaPemilik untuk menyimpan nama pemilik rekening, dan saldo untuk menyimpan jumlah saldo.

```

public Rekening(String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;

    System.out.println(
        "Rekening atas nama " + namaPemilik +
        " berhasil dibuat dengan saldo Rp" + saldo
    );
}

```

Constructor digunakan untuk memberikan nilai awal pada objek rekening ketika rekening dibuat. Data yang dimasukkan berupa nomor rekening, nama pemilik, dan saldo awal. Setelah rekening dibuat, program menampilkan informasi bahwa rekening berhasil dibuat.

```

public void setorTunai(double nominal) {
    if (nominal >= 10000) {
        saldo += nominal;

        System.out.println(
            "Setor tunai Rp" + nominal +
            " berhasil. Saldo saat ini: Rp" + saldo
        );
    } else {
        System.out.println(
            "Gagal, nominal setor minimal Rp10.000!"
        );
    }
}

```

Method setorTunai() digunakan untuk melakukan penyetoran uang. Program memeriksa apakah nominal setor minimal Rp10.000. Jika memenuhi syarat, nominal akan ditambahkan ke saldo. Jika kurang dari Rp10.000, transaksi ditolak.

```

public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println(
            "Gagal, nominal tarik tunai minimal Rp10.000!"
        );
    } else if (nominal > saldo) {
        System.out.println(
            "Gagal, saldo tidak mencukupi!"
        );
    } else {
        saldo -= nominal;

        System.out.println(
            "Tarik tunai Rp" + nominal +
            " berhasil. Saldo saat ini: Rp" + saldo
        );
    }
}

```

Method tarikTunai() digunakan untuk melakukan penarikan uang. Penarikan minimal Rp10.000 dan nominal penarikan tidak boleh lebih besar dari saldo yang tersedia. Dengan demikian, saldo rekening tidak akan menjadi minus.

```

public void cekInformasi() {
    System.out.println("\n--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}

```

Method cekInformasi() digunakan untuk menampilkan informasi rekening yang sedang aktif, seperti nomor rekening, nama pemilik, dan saldo akhir.

2.3 Program mainRekening

2.3.1 Kode Program

```
package pekan1;

import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;

        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {

            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {

                case 1:
```

```
System.out.print("Masukkan No Rekening: ");
String no = input.nextLine();

System.out.print("Masukkan Nama Pemilik: ");
String nama = input.nextLine();

System.out.print("Masukkan Saldo Awal: ");

if (input.hasNextDouble()) {

    double saldo = input.nextDouble();

    akunAktif = new Rekening(no, nama, saldo);
    daftarRekening.add(akunAktif);

} else {

    System.out.println(
        "Error: Saldo awal harus berupa angka!");

    input.next();
}

break;

case 2:

    if (akunAktif == null) {

        System.out.println(
            "Error: Anda belum memiliki rekening!");

    } else {

        System.out.print("Masukkan nominal setor: ");

        if (input.hasNextDouble()) {

            double setor = input.nextDouble();
            akunAktif.setorTunai(setor);
```

```

        } else {

            System.out.println(
                "Error: Nominal harus berupa angka!");

            input.next();
        }

        break;

case 3:

    if (akunAktif == null) {

        System.out.println(
            "Error: Anda belum memiliki rekening!");

    } else {

        System.out.print("Masukkan nominal tarik: ");

        if (input.hasNextDouble()) {

            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);

        } else {

            System.out.println(
                "Error: Nominal harus berupa angka!");

            input.next();
        }

        break;

case 4:

```

```
        if (akunAktif == null) {

            System.out.println(
                "Error: Anda belum membuka rekening!");

        } else {

            akunAktif.cekInformasi();
        }

        break;

    case 5:

        System.out.print("Masukkan nomor rekening: ");
        String nomorCari = input.nextLine();

        boolean ditemukan = false;

        for (Rekening rekening : daftarRekening) {

            if (rekening.nomorRekening.equals(nomorCari)) {

                akunAktif = rekening;
                ditemukan = true;

                System.out.println(
                    "Berhasil mengganti akun ke rekening " +
                    akunAktif.namaPemilik);

                break;
            }
        }

        if (!ditemukan) {
            System.out.println("Rekening tidak ditemukan!");
        }

        break;
```

```

        case 0:

            isRunning = false;

            System.out.println(
                "Sistem ditutup. Terima Kasih!");

            break;

        default:

            System.out.println(
                "Pilihan menu tidak tersedia!");

            break;
    }
}

input.close();
}
}

```

2.3.2 Penjelasan

```

import java.util.ArrayList;
import java.util.Scanner;

```

Scanner digunakan untuk menerima input dari pengguna melalui keyboard. ArrayList digunakan untuk menyimpan beberapa objek rekening sehingga program dapat menangani lebih dari satu rekening.

```

Scanner input = new Scanner(System.in);

```

```

ArrayList<Rekening> daftarRekening = new ArrayList<>();

```

```

Rekening akunAktif = null;

```

input digunakan untuk membaca input pengguna. daftarRekening digunakan untuk menyimpan seluruh rekening yang telah dibuat. Sedangkan akunAktif digunakan untuk menentukan rekening yang sedang digunakan.

```
boolean isRunning = true;
```

```
while (isRunning) {
```

while digunakan agar menu utama terus ditampilkan dan dapat digunakan berulang kali sampai pengguna memilih menu keluar.

```
System.out.println("1. Buka Rekening Baru");  
System.out.println("2. Setor Tunai");  
System.out.println("3. Tarik Tunai");  
System.out.println("4. Cek Informasi Rekening");  
System.out.println("5. Ganti Akun");  
System.out.println("0. Keluar");
```

Menu untuk memberikan beberapa pilihan kepada pengguna, yaitu membuat rekening, melakukan setor tunai, tarik tunai, melihat informasi rekening, mengganti akun, dan keluar dari program.

```
if (!input.hasNextInt()) {  
    System.out.println(  
        "Error: Pilihan menu harus berupa angka!"  
    );  
  
    input.next();  
    continue;  
}
```

Kode tersebut digunakan untuk memastikan pilihan menu yang dimasukkan berupa angka. Jika pengguna memasukkan huruf atau karakter lain, program menampilkan pesan error dan kembali ke menu utama.

```
Rekening rekeningBaru =  
    new Rekening(no, nama, saldo);  
  
daftarRekening.add(rekeningBaru);
```

```
akunAktif = rekeningBaru;
```

Program membuat objek rekening baru menggunakan constructor Rekening. Setelah dibuat, objek tersebut ditambahkan ke dalam ArrayList menggunakan add(). Rekening yang baru dibuat kemudian dijadikan sebagai akunAktif. Dengan menggunakan ArrayList, pengguna dapat memiliki **lebih dari satu rekening** tanpa menghapus rekening sebelumnya.

```
double setor = input.nextDouble();
```

```
akunAktif.setorTunai(setor);
```

Program menerima nominal setor dari pengguna kemudian memanggil method setorTunai() pada rekening yang sedang aktif.

```
double tarik = input.nextDouble();
```

```
akunAktif.tarikTunai(tarik);
```

Program menerima nominal penarikan kemudian memanggil method tarikTunai(). Pemeriksaan minimal Rp10.000 dan saldo yang mencukupi dilakukan di class Rekening.

```
akunAktif.cekInformasi();
```

Kode tersebut digunakan untuk menampilkan informasi dari rekening yang sedang aktif.

```
for (Rekening rekening : daftarRekening) {  
    if (rekening.nomorRekening.equals(nomorCari)) {  
        akunAktif = rekening;  
        ditemukan = true;  
        break;  
    }  
}
```

Bagian ini digunakan untuk mencari rekening berdasarkan nomor rekening di dalam ArrayList. Program memeriksa setiap rekening menggunakan for. Jika nomor rekening sesuai, rekening tersebut dijadikan sebagai akunAktif.

case 0:

```
isRunning = false;
```

```
System.out.println(  
    "Sistem ditutup. Terima Kasih!"  
);
```

```
break;
```

Jika kita memilih menu 0, nilai isRunning diubah menjadi false, sehingga perulangan berhenti dan program ditutup.

2.4 Output

```
=== SISTEM PERBANKAN MINI ===  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu: 1  
Masukkan No Rekening: 1  
Masukkan Nama Pemilik: nafi  
Masukkan Saldo Awal: 20000  
Rekening atas nama nafi berhasil dibuat dengan saldo Rp20000.0  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu: 2  
Masukkan nominal setor: 10000  
Setor tunai Rp10000.0 berhasil. Saldo saat ini: Rp30000.0  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu: 3  
Masukkan nominal tarik: 5000  
Gagal, nominal tarik tunai minimal Rp10.000!
```

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar

Pilih menu: 1

Masukkan No Rekening: 2

Masukkan Nama Pemilik: ghifari

Masukkan Saldo Awal: 30000

Rekening atas nama ghifari berhasil dibuat dengan saldo Rp30000.0

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar

Pilih menu: 2

Masukkan nominal setor: 10000

Setor tunai Rp10000.0 berhasil. Saldo saat ini: Rp40000.0

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar

Pilih menu: 3

Masukkan nominal tarik: 50000

Gagal, saldo tidak mencukupi, saldo Anda : Rp40000.0

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar

Pilih menu: 5

Masukkan nomor rekening: 1

Berhasil mengganti akun ke rekening nafi

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar

Pilih menu: 5

Masukkan nomor rekening: 2

Berhasil mengganti akun ke rekening ghifari

KESIMPULAN

Rekening.java berfungsi sebagai class yang mengatur **data dan operasi rekening**, seperti membuat rekening, setor tunai, tarik tunai, dan melihat informasi rekening.

Sedangkan **Main.java** berfungsi sebagai **program utama** yang mengatur interaksi dengan pengguna melalui menu. Penggunaan `ArrayList<Rekening>` memungkinkan sistem menyimpan **banyak rekening**, sedangkan `akunAktif` digunakan untuk menentukan rekening yang sedang digunakan.